Merge Sort Visualizer

1. Introduction

This project is a **Merge Sort Visualizer** implemented using **Python and Pygame**. It visually demonstrates how the Merge Sort algorithm sorts an array of integers by highlighting the merging process in real time.

2. Features

- **Graphical Representation:** Displays sorting steps using colored bars.
- **Interactive Controls:**
  - Press **ENTER** to start sorting.
  - Press **R** to reset the array.
  - Close the window to exit.
- **Color-coded Sorting:**
  - **Cyan:** Default color for unsorted elements.
  - **Red:** Elements being merged.
  - **Green:** Sorted elements.

3. Prerequisites

Software Requirements:

- Python 3.x
- Pygame Library

Installation Guide:

1. Install Python from python.org.
2. Install Pygame using pip:
3. `pip install pygame`

4. Code Explanation

Importing Required Libraries

```
import pygame
import random
```

- `pygame`: Used for graphical visualization.
- `random`: Generates a random array for sorting.

Initializing Pygame and Screen Setup

```
pygame.font.init()
width, height = 800, 600
```

```
screen = pygame.display.set_mode((width, height))
pygame.display.set_caption("Merge Sort Visualizer by edSlash Coding Hub")
```

- Initializes Pygame's font system.
- Creates a window of size **800x600**.
- Sets the title of the window.

Color Definitions

```
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
CYAN = (0, 204, 204)
RED = (255, 0, 0)
GREEN = (0, 255, 0)
```

- Defines colors using **RGB tuples**.

Array Initialization

```
size = 25
array = [random.randint(10, 400) for _ in range(size)]
colors = [CYAN] * size
```

- Generates an array of 25 random integers between **10 and 400**.
- Initializes all bars with the **Cyan** color.

Font Setup

```
font = pygame.font.SysFont("comicsans", 30)
```

- Loads the **Comic Sans** font with size **30** for display messages.

Drawing Function

```
def draw(message="Press ENTER to Sort | Press R to Reset"):
    screen.fill(WHITE)
    text = font.render(message, True, BLACK)
    screen.blit(text, (20, 20))
    bar_width = width // size
    for i in range(size):
        pygame.draw.rect(screen, colors[i],
                         (i * bar_width, height - array[i], bar_width - 2,
array[i]))
    pygame.display.update()
```

- Clears the screen and displays an **instruction message**.
- Iterates over the `array` and draws bars of heights proportional to their values.
- Calls `pygame.display.update()` to reflect changes on the screen.

Merge Sort Implementation

```
def merge(low, mid, high):
    left_part = array[low:mid+1]
```

```
    right_part = array[mid+1:high+1]
    left_index, right_index, sorted_index = 0, 0, low

    while left_index < len(left_part) and right_index < len(right_part):
        pygame.event.pump()
        colors[sorted_index] = RED  # Mark elements being merged
        draw()
        pygame.time.delay(50)

        if left_part[left_index] <= right_part[right_index]:
            array[sorted_index] = left_part[left_index]
            left_index += 1
        else:
            array[sorted_index] = right_part[right_index]
            right_index += 1

        sorted_index += 1

    while left_index < len(left_part):
        pygame.event.pump()
        array[sorted_index] = left_part[left_index]
        colors[sorted_index] = RED
        draw()
        pygame.time.delay(50)
        left_index += 1
        sorted_index += 1

    while right_index < len(right_part):
        pygame.event.pump()
        array[sorted_index] = right_part[right_index]
        colors[sorted_index] = RED
        draw()
        pygame.time.delay(50)
        right_index += 1
        sorted_index += 1

    for i in range(low, high + 1):
        colors[i] = GREEN
    draw()

def merge_sort(low, high):
    if low < high:
        mid = (low + high) // 2
        merge_sort(low, mid)
        merge_sort(mid + 1, high)
        merge(low, mid, high)

    if low == 0 and high == size - 1:
        colors[:] = [GREEN] * size
        draw("Sorting Completed!")
        pygame.time.delay(1000)
```

- Recursively divides the array into smaller halves.
- Merges them back while visualizing the process.
- Uses **color changes** to highlight merging progress.
- Calls `pygame.event.pump()` to prevent window freezing.
- Delays sorting steps for smooth visualization.

Main Game Loop

```
running = True
while running:
    screen.fill(WHITE)
    draw()

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_r:
                array = [random.randint(10, 400) for _ in range(size)]
                colors = [CYAN] * size
            if event.key == pygame.K_RETURN:
                merge_sort(0, size - 1)
pygame.quit()
```

- Runs a **continuous loop** to listen for user inputs.
- **`R` key:** Resets the array.
- **`ENTER` key:** Starts the sorting process.
- **QUIT event:** Closes the program.

5. Sample Output

Upon running the script, a window appears with 25 bars representing randomly generated numbers. As the sorting progresses:

- Elements being merged turn **Red**.
- Sorted elements turn **Green**.
- The final sorted array remains **Green** with a "Sorting Completed!" message displayed.

6. Future Enhancements

- Add more sorting algorithms like **Heap Sort and Radix Sort**.
- Introduce **step-by-step sorting mode** for better learning.
- Improve **UI design** and add user-defined array input.

7. Conclusion

This project successfully visualizes **Merge Sort** using **Pygame**, making the sorting process easy to understand. It is an excellent learning tool for those new to sorting algorithms and Python game development.

8. References

- Pygame Documentation
- Merge Sort Explanation